

# C++ Multithreaded HTTP Server

## Core + Resume-Worthy Build Specification

### Project Goal

Build a **multithreaded HTTP/1.1 server from scratch in C++** using POSIX TCP sockets and the C++ standard threading/concurrency library.

The project will start as a simple TCP server and progressively evolve into a properly structured HTTP server capable of handling multiple clients concurrently through a custom thread pool.

The final Core + Resume-Worthy version will demonstrate:

- C++ systems programming
- TCP networking
- HTTP fundamentals
- object-oriented design
- concurrency
- thread synchronization
- thread pools
- request routing
- error handling
- graceful shutdown
- automated testing
- performance benchmarking
- clean software architecture
- CMake and Git

The objective is **not** to build the largest possible server.

The objective is to build a relatively compact system that you understand deeply enough to explain every important design decision in an interview.

---

## 1. Final Scope

For now, the project will contain exactly these major capabilities:

## CORE

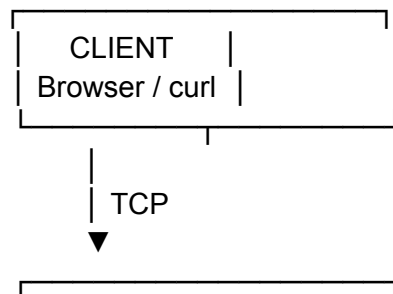
- TCP socket server
- HTTP request parsing
- HTTP response generation
- GET requests
- POST requests
- Request body handling
- Routing
- Error handling
- Multithreading
- Custom thread pool
- Thread-safe task queue
- Shared-state synchronization
- CMake build system
- Unit + integration tests

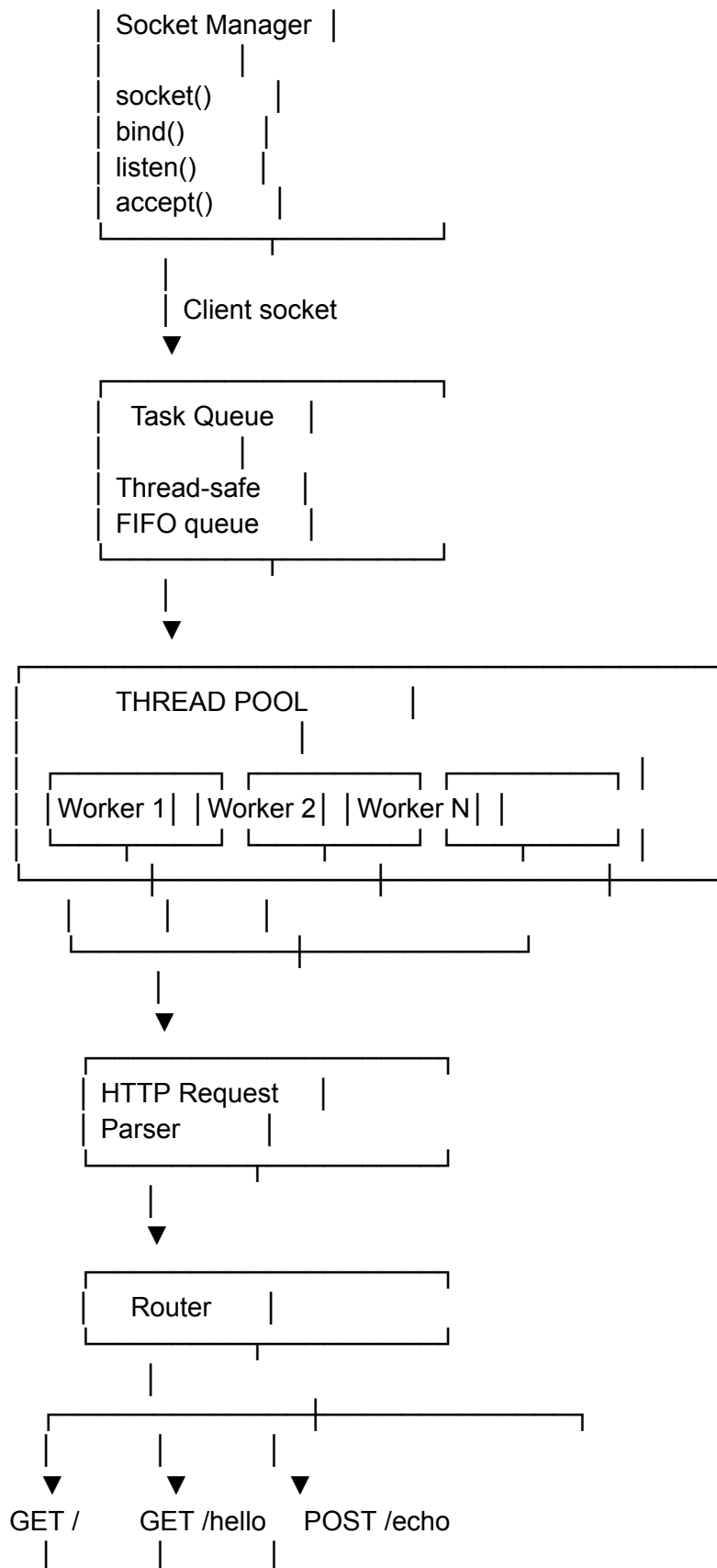
## RESUME-WORTHY

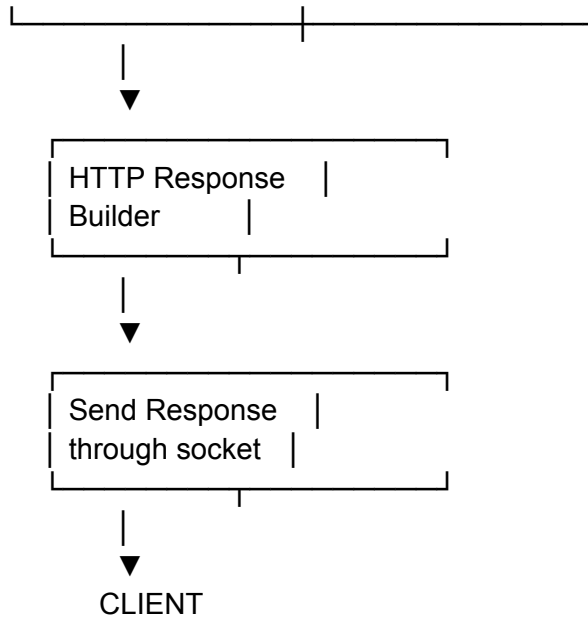
- Graceful shutdown
  - Structured logging
  - Runtime metrics
  - Load testing
  - Performance benchmarking
  - Worker-count comparison
  - Sanitizer-based debugging
  - Strong technical documentation
- 

## 2. What the Final System Should Look Like

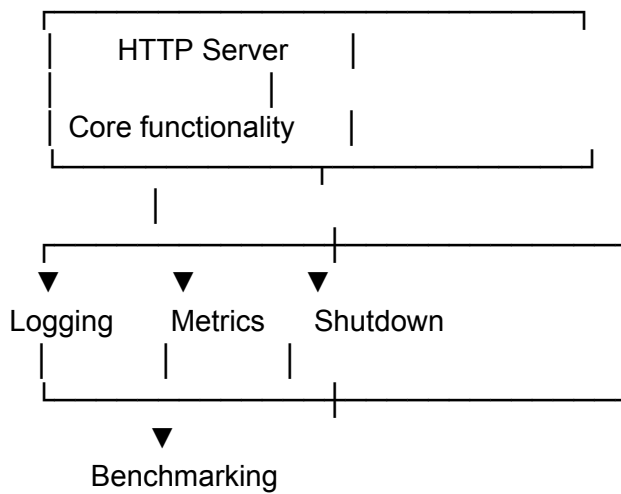
The target architecture is:







The resume-worthy additions sit around this core:



---

## 3. Development Philosophy

Build the system **from the bottom up**.

Do not start by writing the final architecture.

Instead:

TCP

↓  
HTTP  
↓  
Routing  
↓  
Single-threaded server  
↓  
Concurrency  
↓  
Thread pool  
↓  
Synchronization  
↓  
Robustness  
↓  
Testing  
↓  
Logging + Metrics  
↓  
Benchmarking

Every phase should leave you with a working system.

This is important because when something breaks, you know which layer introduced the problem.

---

## 4. Phase 0 — Development Environment

Before writing server code, establish the development environment.

### Required tools

#### Language

C++17 or C++20.

Use C++17 if you want maximum simplicity and compatibility.

---

#### Compiler

Use either:

g++

or:

clang++

On your Mac, Clang is perfectly suitable.

---

## **Build system**

Use:

CMake

---

## **Version control**

Use:

Git

GitHub

---

## **Testing**

Use:

GoogleTest

---

## **Debugging**

Become comfortable with:

gdb

or:

lldb

depending on your environment.

---

## Sanitizers

Use:

AddressSanitizer

ThreadSanitizer

where supported by your compiler/platform.

---

## Load testing

Use:

wrk

or another HTTP benchmarking tool.

---

# 5. Phase 1 — Basic TCP Server

## Objective

Understand the lowest-level communication layer before thinking about HTTP.

The server should:

1. create a TCP socket
  2. bind it to a port
  3. listen for connections
  4. accept a client
  5. receive data
  6. send data
  7. close the connection
- 

# 6. TCP Mental Model

The server performs:

```
socket()
↓
bind()
↓
listen()
↓
accept()
↓
recv()
↓
send()
↓
close()
```

The client performs approximately:

```
socket()
↓
connect()
↓
send()
↓
recv()
↓
close()
```

The critical idea is:

TCP gives us a reliable byte stream. TCP does not understand HTTP.

Your application will interpret those bytes as HTTP.

---

## 7. Phase 1 Functional Requirements

### FR-01

The server must create a TCP socket.

### FR-02

The server must bind to a configurable port.



For the first version:

8080

is sufficient.

### **FR-03**

The server must listen for incoming connections.

### **FR-04**

The server must accept client connections.

### **FR-05**

The server must receive bytes from the client.

### **FR-06**

The server must send a response.

### **FR-07**

The server must close the connection safely.

---

## **8. First Milestone**

You should be able to run:

```
./server
```

and then:

```
curl localhost:8080
```

and receive a response.

Do not move on until this works reliably.

---

## 9. Phase 2 — Understand and Generate HTTP Responses

Now introduce HTTP.

A basic response looks like:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 12
```

Hello World!

The structure is:

```
Status Line
  ↓
Headers
  ↓
Blank Line
  ↓
Body
```

---

## 10. HttpResponse

Create a dedicated representation:

```
class HttpResponse {
private:
    int statusCode;
    std::string statusMessage;

    std::unordered_map<std::string, std::string> headers;

    std::string body;

public:
    void setStatus(int code,
                  const std::string& message);
```

```
void setHeader(const std::string& key,
               const std::string& value);

void setBody(const std::string& body);

std::string serialize() const;
};
```

The important design principle is:

The HTTP response should be represented as an object before being converted into raw bytes.

---

## 11. Required Status Codes

Initially support:

```
200 OK
201 Created
400 Bad Request
404 Not Found
405 Method Not Allowed
500 Internal Server Error
```

You do not need every HTTP status code.

---

## 12. Phase 3 — HTTP Request Parser

This is one of the most important components.

A TCP connection may give you bytes like:

```
GET /hello HTTP/1.1\r\n
Host: localhost:8080\r\n
User-Agent: curl\r\n
Accept: */*\r\n
\r\n
```

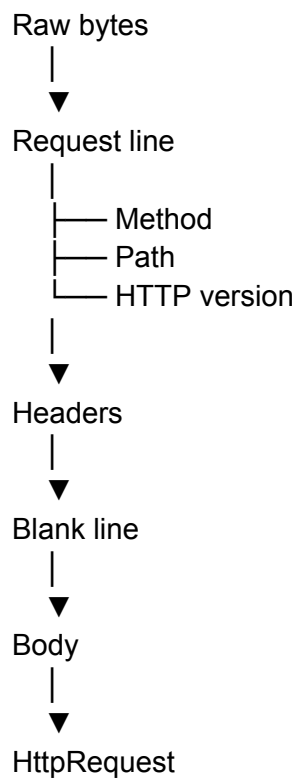
Your parser should convert this into:

```
struct HttpRequest {  
    std::string method;  
    std::string path;  
    std::string version;  
  
    std::unordered_map<  
        std::string,  
        std::string  
    > headers;  
  
    std::string body;  
};
```

---

## 13. Parsing Pipeline

The parser should conceptually perform:



## 14. Methods for the Core Version

Implement:

GET  
POST

Do not implement every HTTP method yet.

---

## 15. Request Body

For POST requests, support:

Content-Length

so the server knows how much body data it needs to read.

Example:

```
POST /echo HTTP/1.1
Content-Type: text/plain
Content-Length: 5
```

hello

The resulting request should contain:

```
method = POST
path   = /echo
body   = hello
```

---

## 16. Parser Validation

The parser must reject malformed requests.

Examples:

- Invalid request line
- Missing HTTP version
- Malformed headers
- Invalid Content-Length
- Request too large

Incomplete request

These should result in an appropriate error response rather than crashing the server.

---

## 17. Phase 4 — Router

The router maps:

HTTP method + path

to:

handler

For example:

```
GET /  
GET /hello  
POST /echo
```

---

## 18. Handler Abstraction

A useful abstraction is:

```
using Handler =  
    std::function<HttpResponse(  
        const HttpRequest&  
    )>;
```

Then routes can conceptually be registered as:

```
router.get("/", homeHandler);
```

```
router.get("/hello", helloHandler);
```

```
router.post("/echo", echoHandler);
```

---

# 19. Router Requirements

## FR-08

The router must allow routes to be registered.

## FR-09

The router must match HTTP method.

## FR-10

The router must match request path.

## FR-11

The router must invoke the appropriate handler.

## FR-12

Unknown routes must return:

404 Not Found

## FR-13

Known routes with unsupported methods must return:

405 Method Not Allowed

---

# 20. Initial Routes

Implement these three routes.

## GET /

Returns:

Hello from C++ HTTP Server

## GET /hello

Returns:

Hello World!

## **POST /echo**

Returns the request body.

For example:

POST /echo

hello

returns:

hello

These routes are intentionally simple.

The point is testing the architecture, not building a complex application.

---

# **21. Phase 5 — Complete Single-Threaded HTTP Server**

At this stage the complete pipeline should work:





HttpResponse



Serialize



Socket



Client

Everything should still be single-threaded.

This is a critical milestone.

---

## 22. Why Build Single-Threaded First?

Concurrency makes debugging much harder.

Suppose you immediately implement:

TCP

+

HTTP

+

Router

+

Threads

and something crashes.

You now have several possible causes.

Instead:

First:

TCP works

Then:

HTTP works

Then:

Routing works

Then:

Concurrency is introduced

This isolates complexity.

---

## 23. Phase 6 — Introduce Concurrency

Now make the server capable of processing multiple clients simultaneously.

The naïve architecture is:

```
while (running) {  
    int client = accept(...);  
  
    std::thread(  
        handleClient,  
        client  
    ).detach();  
}
```

This gives each connection its own thread.

It works, but it is not the final design.

---

## 24. Why Not One Thread Per Connection?

Imagine:

100 clients  
→ 100 threads

This may be acceptable.

But:

10,000 clients  
→ 10,000 threads

is problematic.

Threads consume:

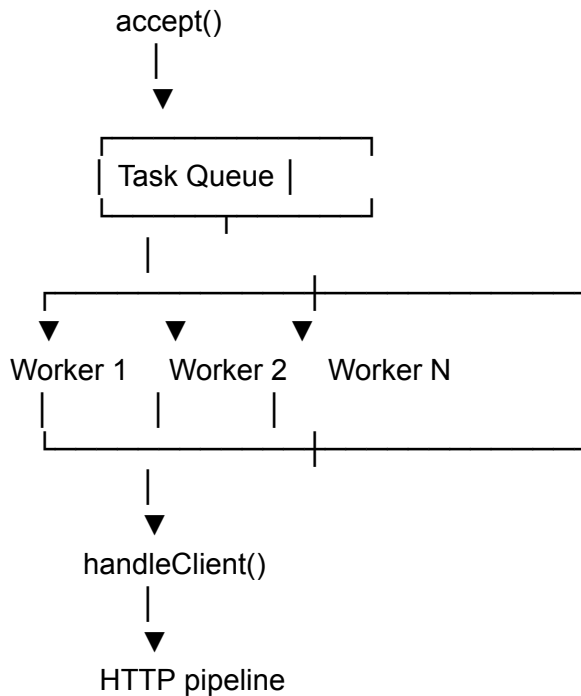
- memory
- scheduling resources
- CPU time
- context-switching overhead

Therefore we need a bounded number of worker threads.

---

## 25. Phase 7 — Build the Thread Pool

The final concurrency architecture is:



---

## 26. Thread Pool Responsibilities

The thread pool must:

1. create worker threads
2. maintain a task queue
3. allow tasks to be submitted

4. allow workers to retrieve tasks
  5. sleep when no tasks exist
  6. wake when work arrives
  7. stop safely
  8. join worker threads
- 

## 27. Thread Pool Data Structures

A basic implementation needs:

```
std::vector<std::thread> workers;
```

```
std::queue<std::function<void()>> tasks;
```

```
std::mutex queueMutex;
```

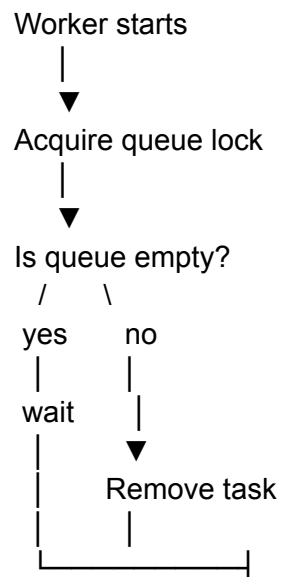
```
std::condition_variable condition;
```

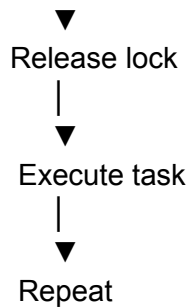
```
bool stopping;
```

---

## 28. Worker Algorithm

Conceptually:





The worker should **not hold the queue mutex while executing the task**.

This is an important design decision.

---

## 29. Why a Condition Variable?

Without a condition variable, workers might continuously poll:

Is there work?  
Is there work?  
Is there work?  
Is there work?

This wastes CPU.

Instead:

No task  
↓  
Worker sleeps  
↓  
New task arrives  
↓  
condition\_variable.notify\_one()  
↓  
Worker wakes  
↓  
Processes task

---

## 30. Thread-Safe Task Queue

The queue is shared between:

Main thread  
+  
Worker threads

Therefore access must be synchronized.

Conceptually:

```
{  
    std::lock_guard<std::mutex> lock(queueMutex);  
  
    tasks.push(task);  
}
```

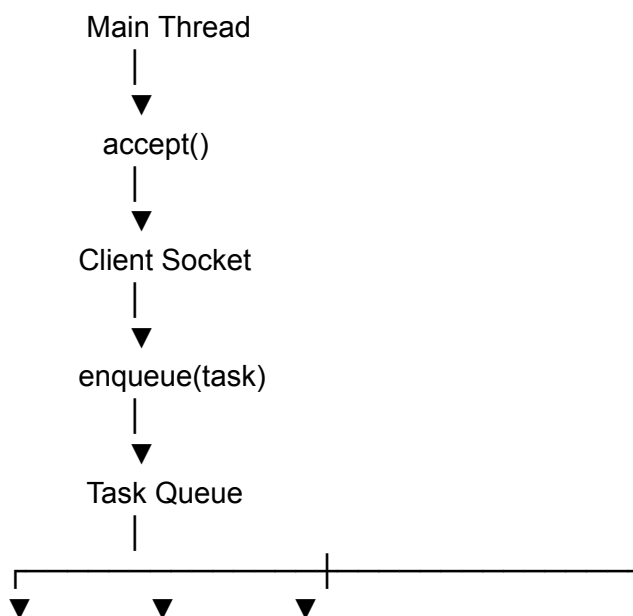
```
condition.notify_one();
```

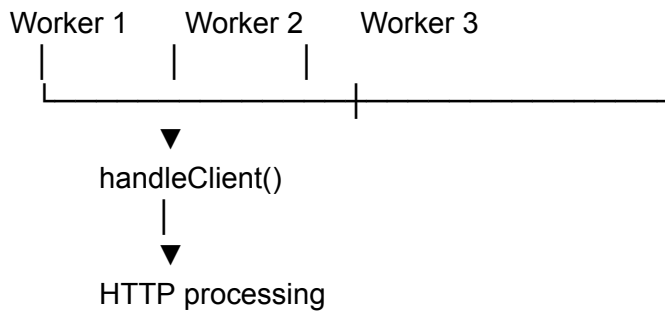
Workers similarly acquire the lock before removing a task.

---

## 31. Phase 8 — Integrate Thread Pool With Server

The server now becomes:





## 32. Important Design Decision

The main thread should primarily:

- accept connections
- enqueue work
- coordinate lifecycle

Worker threads should:

- read request
- parse request
- route request
- execute handler
- generate response
- send response

This separation keeps responsibilities clear.

## 33. Phase 9 — Thread-Safe Shared State

Now introduce shared server state.

For example:

```
requests_total
errors_total
active_connections
```

These values may be accessed by multiple worker threads.

---

## 34. Atomic Variables

For simple counters:

```
std::atomic<uint64_t> requestsTotal;
```

is appropriate.

For example:

```
requestsTotal.fetch_add(1);
```

The important concept is:

Multiple threads can update the counter without creating a data race.

---

## 35. Mutex-Protected State

Not every shared structure can be represented by a simple atomic.

For example:

```
std::unordered_map<std::string, int>
```

may require a mutex if multiple threads modify it.

The general rule:

Simple independent counter

↓  
atomic

Complex shared data structure

↓  
mutex

---

## 36. Phase 10 — Graceful Shutdown



The server must not simply terminate when the user presses:

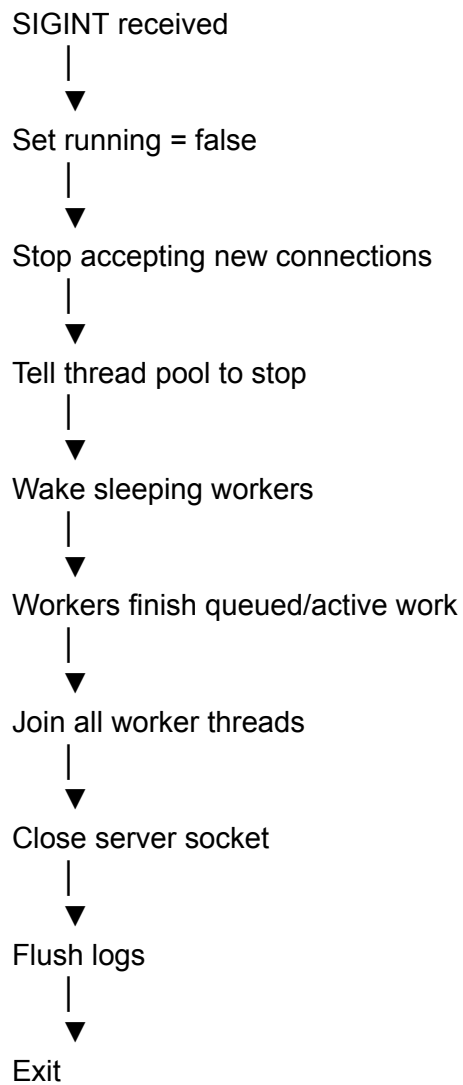
Ctrl+C

Instead, implement graceful shutdown.

---

## 37. Shutdown Sequence

The desired sequence is:



## 38. Why Graceful Shutdown Matters

Without graceful shutdown:

- connections may be abruptly terminated
- resources may not be released
- worker threads may remain alive
- logs may be lost
- sockets may remain open

Graceful shutdown demonstrates that you understand **lifecycle management**, not just normal execution.

---

## 39. Phase 11 — Error Handling

The server must handle expected failures without crashing.

---

### Invalid HTTP request

Return:

400 Bad Request

---

### Unknown route

Return:

404 Not Found

---

### Unsupported method

Return:

405 Method Not Allowed

---

## Handler failure

Return:

500 Internal Server Error

---

## Oversized request

Return:

413 Payload Too Large

---

# 40. Request Limits

Implement at least:

`MAX_REQUEST_SIZE`

`MAX_HEADER_SIZE`

`MAX_BODY_SIZE`

These can initially be constants.

For example:

`MAX_REQUEST_SIZE = 1 MB`

The exact values are less important than understanding why limits exist.

Without limits, clients can potentially consume excessive memory.

---

# 41. Phase 12 — Structured Logging

Once the core server works, introduce logging.

Example:

`[INFO] 2026-09-25 21:10:31`

`GET /hello`

status=200  
latency=2ms  
worker=3

Error:

[ERROR] 2026-09-25 21:10:45  
POST /echo  
status=400  
reason="invalid Content-Length"

---

## 42. Logging Requirements

The logger should record:

- timestamp
- log level
- HTTP method
- path
- status code
- latency
- worker ID where practical
- error reason when applicable

At minimum support:

INFO  
WARNING  
ERROR

DEBUG can be added if useful.

---

## 43. Why Logging Is Resume-Worthy

Logging isn't impressive because logging itself is difficult.

It is valuable because it demonstrates that you're thinking about:

- production debugging
- observability

- failure diagnosis
  - operational behavior
- 

## 44. Phase 13 — Metrics

Add server-level metrics.

At minimum track:

total requests  
successful requests  
failed requests  
active connections  
total errors

Also track latency.

---

## 45. Latency Measurement

Measure:

request received  
↓  
handler begins  
↓  
response generated  
↓  
response sent

For a request:

start = t1  
end = t2

latency is:

latency = t2 - t1

Use a monotonic clock such as:

std::chrono::steady\_clock

for duration measurements.

---

## 46. Latency Percentiles

Eventually calculate:

p50

p95

p99

### **p50**

Half of requests are faster than this value.

### **p95**

95% of requests are faster than this value.

### **p99**

99% of requests are faster than this value.

These are more informative than average latency alone.

---

## 47. Metrics Endpoint

Add:

GET /metrics

Example response:

```
{  
  "requests_total": 15231,  
  "errors_total": 42,  
  "active_connections": 7,  
  "average_latency_ms": 4.3
```

```
}
```

The exact representation can be simple.

The important thing is that the server can expose its internal state.

---

## 48. Phase 14 — Automated Testing

Testing is part of the project, not an optional final step.

Use GoogleTest for unit tests.

Suggested structure:

```
tests/  
├── unit/  
│   ├── HttpParserTest.cpp  
│   ├── HttpResponseTest.cpp  
│   ├── RouterTest.cpp  
│   └── ThreadPoolTest.cpp  
└── integration/  
    └── ServerTest.cpp
```

---

## 49. HTTP Parser Tests

Test valid requests:

GET / HTTP/1.1

GET /hello HTTP/1.1

POST /echo HTTP/1.1

Test headers.

Test bodies.

Test malformed requests.

Test missing fields.

Test invalid Content-Length.

---

## 50. HTTP Response Tests

Verify:

status line  
headers  
Content-Length  
body

Example:

Input body:  
Hello

Expected:  
Content-Length: 5

---

## 51. Router Tests

Test:

GET /  
GET /hello  
POST /echo

and:

GET /does-not-exist

which should produce 404.

Test method mismatch:

POST /

when only GET is registered.

---



## 52. Thread Pool Tests

Verify:

- tasks execute
  - multiple tasks execute
  - multiple workers can process tasks
  - queue remains consistent
  - workers sleep when there is no work
  - shutdown terminates workers
  - tasks aren't lost unexpectedly
- 

## 53. Integration Tests

Start the real server.

Send actual HTTP requests.

Verify:

GET /  
GET /hello  
POST /echo  
GET /unknown

Check:

status  
headers  
body

This verifies that the individual components work correctly together.

---

## 54. Phase 15 — Concurrency Testing

This is especially important because concurrency is one of the project's primary goals.

Send multiple requests simultaneously.

For example:

1000 requests  
50 concurrent clients

Check:

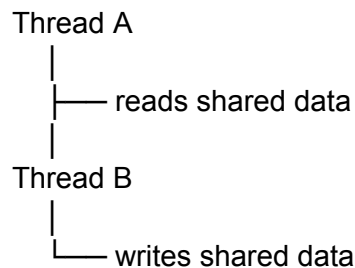
server does not crash  
responses remain valid  
request count is correct  
shared state is not corrupted

---

## 55. Race Detection

Use ThreadSanitizer where supported.

The purpose is to find cases where:



without appropriate synchronization.

A project specifically focused on concurrency should demonstrate that concurrency was tested rather than merely implemented.

---

## 56. Memory Safety

Use AddressSanitizer where supported.

Look for:

- buffer overflows
- use-after-free
- invalid memory access

- memory leaks

This is especially useful when dealing with:

sockets

buffers

threads

dynamic memory

---

## 57. Phase 16 — Load Testing

Now the project becomes genuinely resume-worthy.

We want to answer:

How does the server behave under concurrent load?

Use:

wrk

or another HTTP benchmarking tool.

---

## 58. Benchmark Setup

Choose a fixed endpoint:

GET /hello

Then vary the number of workers.

For example:

1 worker

2 workers

4 workers

8 workers

Keep the benchmark conditions consistent.

---

## 59. Measurements

Record:

Requests/sec  
Average latency  
p50 latency  
p95 latency  
p99 latency  
Error rate

Do not make up these numbers.

Use your actual machine and actual measurements.

---

## 60. Benchmark Experiment

The final README should eventually contain a table similar to:

| Workers | Requests/se<br>c | p50       | p95       | p99       | Errors |
|---------|------------------|-----------|-----------|-----------|--------|
| 1       | actual           | actua<br> | actua<br> | actua<br> | actual |
| 2       | actual           | actua<br> | actua<br> | actua<br> | actual |
| 4       | actual           | actua<br> | actua<br> | actua<br> | actual |
| 8       | actual           | actua<br> | actua<br> | actua<br> | actual |

The important part is not necessarily getting the highest number.

The important part is being able to explain **why performance changes as worker count changes**.

---

# 61. What You Should Analyze

Ask:

**Does doubling workers double throughput?**

Probably not indefinitely.

Why?

Potential bottlenecks include:

- CPU
- network
- system calls
- mutex contention
- memory
- context switching

---

**At what point does adding workers stop helping?**

Measure it.

---

**Does the p95 latency improve?**

Not necessarily.

More workers can sometimes increase contention.

---

**Is the server CPU-bound or I/O-bound?**

Your workload and benchmark can help answer this.

---

## 62. Phase 17 — CMake

Your project should use a proper CMake build.

The build should support:

```
mkdir build
cd build
cmake ..
make
```

and:

```
./server
```

Tests should also be buildable through CMake.

For example:

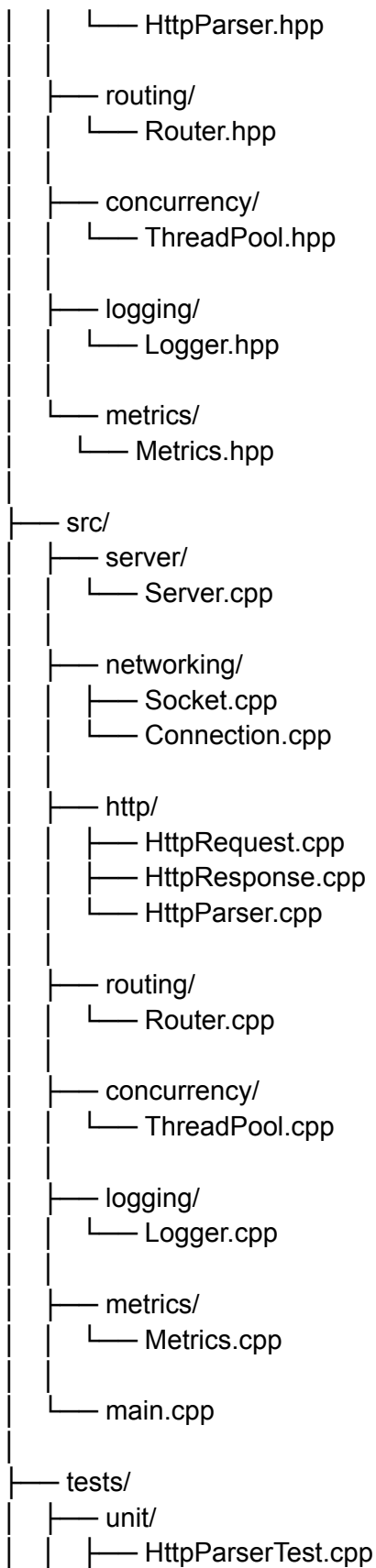
```
ctest
```

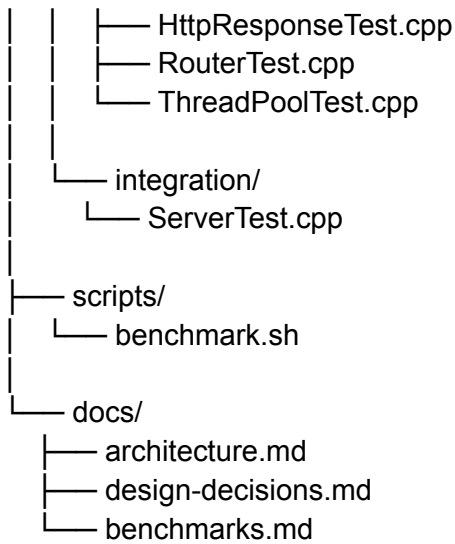
---

## 63. Suggested Project Structure

Use:

```
cpp-http-server/
|
|— CMakeLists.txt
|— README.md
|— .gitignore
|
|— include/
|   |— server/
|   |   |— Server.hpp
|   |
|   |— networking/
|   |   |— Socket.hpp
|   |   |— Connection.hpp
|   |
|   |— http/
|   |   |— HttpRequest.hpp
|   |   |— HttpResponse.hpp
```





Do not worry if the exact structure evolves.

The important thing is separation of responsibility.

---

## 64. Component Responsibilities

### Server

Responsible for:

- initialization
- lifecycle
- accept loop
- shutdown

It should coordinate other components rather than implementing everything itself.

---

### Socket

Responsible for:

- socket creation
- bind



- listen
- accept
- send
- receive
- close

It should not understand HTTP.

---

## HttpParser

Responsible for:

- parsing request line
- parsing headers
- parsing body
- validating HTTP input

It should not know anything about threads or routing.

---

## HttpRequest

Represents parsed request data.

---

## HttpResponse

Represents response data and serialization.

---

## Router

Responsible for:

method + path



handler

---

## ThreadPool

Responsible for:

- workers
- task queue
- synchronization
- worker lifecycle

It should not know that tasks happen to be HTTP requests.

---

## Logger

Responsible for structured logging.

---

## Metrics

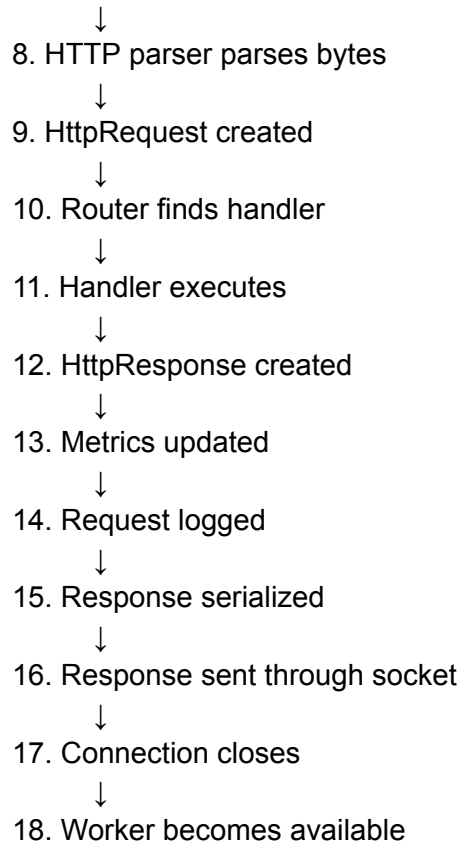
Responsible for collecting server statistics.

---

# 65. Complete Request Lifecycle

You should understand this sequence completely:

1. Client opens TCP connection
- ↓
2. Server accept() receives connection
- ↓
3. Server creates request-handling task
- ↓
4. Task is added to thread pool queue
- ↓
5. Worker wakes up
- ↓
6. Worker retrieves task
- ↓
7. Worker reads socket data

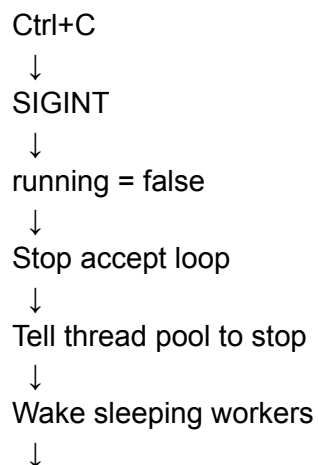


You should eventually be able to draw this without looking at the documentation.

---

## 66. Complete Shutdown Lifecycle

Similarly:



Workers finish active tasks



Workers exit



Join workers



Close server socket



Flush logs



Exit

---

## 67. Core Data Structures

You should deliberately use standard data structures where appropriate.

### Router

Potentially:

`std::unordered_map`

---

### Thread Pool

`std::queue`

for pending tasks.

---

### Worker collection

`std::vector<std::thread>`

---

### Headers

`std::unordered_map<std::string, std::string>`

---

### Shared counters

`std::atomic<uint64_t>`

---

## 68. C++ Concepts You Will Practice

This project should strengthen your understanding of:

- Classes
- Structs
- Encapsulation
- Constructors/destructors
- RAII
- STL
- `std::thread`
- `std::mutex`
- `std::lock_guard`
- `std::unique_lock`
- `std::condition_variable`
- `std::atomic`
- `std::function`
- Lambdas
- Smart pointers
- Move semantics
- Exceptions

You don't need to force every C++ feature into the project.

Use the language features because they solve real problems.

---

## 69. Concurrency Concepts You Must Understand

By the end, you should be able to explain:

### Race condition

Two threads access shared state concurrently and at least one modifies it without adequate synchronization.

---

## Mutex

Provides mutual exclusion around critical sections.

---

## Condition variable

Allows threads to sleep until a condition becomes true.

---

## Atomic

Provides atomic operations on suitable individual variables.

---

## Deadlock

Threads wait indefinitely for resources held by each other.

---

## Thread pool

Maintains a bounded set of workers that repeatedly consume tasks from a queue.

---

# 70. Networking Concepts You Must Understand

Before calling the project complete, you should understand:

IP address  
Port  
TCP

Socket  
Client  
Server  
File descriptor  
Blocking I/O  
listen()  
accept()  
recv()  
send()

You don't need to become a networking expert.

But you should understand what every networking call in your project does.

---

## 71. HTTP Concepts You Must Understand

Know:

HTTP request  
HTTP response  
request line  
status line  
headers  
body  
GET  
POST  
Content-Length  
Content-Type  
404  
400  
405  
500

You should understand that HTTP is being transported as bytes through TCP.

---

## 72. Security Requirements for This Version

Keep security limited but meaningful.

The server should:

1. limit request size
2. validate Content-Length
3. reject malformed HTTP
4. avoid unsafe buffer operations
5. handle unexpected client disconnects
6. avoid crashing on malformed input

Static-file path traversal is **not required yet** because static file serving is outside the current scope.

---

## 73. Resume-Worthy Requirements

Before calling this project “resume-ready,” it should satisfy these requirements:

- ✓ C++17/20
- ✓ POSIX TCP sockets
- ✓ HTTP request parsing
- ✓ HTTP response generation
- ✓ GET
- ✓ POST
- ✓ Routing
- ✓ Custom thread pool
- ✓ Mutex
- ✓ Condition variable
- ✓ Atomic shared state
- ✓ Graceful shutdown
- ✓ Structured logging
- ✓ Metrics
- ✓ Unit tests
- ✓ Integration tests
- ✓ Concurrency tests
- ✓ Sanitizer testing
- ✓ Load testing
- ✓ Benchmark results
- ✓ CMake
- ✓ Clean Git history
- ✓ Strong README



You don't need every possible web-server feature.

These are enough.

---

## 74. Definition of Done

The Core + Resume-Worthy version is complete when:

### **Networking**

The server accepts TCP connections and communicates reliably.

### **HTTP**

It correctly handles basic GET and POST requests.

### **Routing**

Different routes invoke different handlers.

### **Concurrency**

Multiple requests can be processed simultaneously.

### **Thread Pool**

A bounded number of worker threads consume tasks from a synchronized queue.

### **Synchronization**

Shared state is protected correctly.

### **Reliability**

Malformed requests do not crash the server.

### **Shutdown**

Ctrl+C causes an orderly shutdown.

### **Logging**

Requests and errors can be inspected.

## **Metrics**

Basic request and latency statistics are available.

## **Testing**

Core components have automated tests.

## **Benchmarking**

Different worker configurations have been measured.

## **Documentation**

The architecture and design decisions are clearly explained.

---

# **75. Suggested Build Schedule**

Do not treat these as rigid calendar deadlines. Each phase should be completed and understood before moving on.

## **Stage 1 — Networking Foundation**

### **Work**

TCP socket  
bind  
listen  
accept  
recv  
send  
close

### **Deliverable**

A server that responds to a TCP client.

---

## Stage 2 — HTTP Foundation

### Work

HttpRequest  
HttpResponse  
HTTP parser  
HTTP serializer  
status codes  
headers

### Deliverable

A single-threaded HTTP server that responds correctly to curl.

---

## Stage 3 — Routing

### Work

Router  
GET /  
GET /hello  
POST /echo  
404  
405

### Deliverable

Multiple HTTP endpoints.

---

## Stage 4 — Thread Pool

### Work

std::thread  
std::mutex  
std::queue  
std::condition\_variable

ThreadPool

### **Deliverable**

Multiple simultaneous clients can be handled concurrently.

---

## **Stage 5 — Synchronization + Robustness**

### **Work**

atomic counters  
mutex-protected state  
request limits  
error handling

### **Deliverable**

A stable concurrent server.

---

## **Stage 6 — Graceful Shutdown**

### **Work**

SIGINT  
running flag  
worker shutdown  
thread joining  
socket cleanup

### **Deliverable**

Clean server lifecycle.

---

## **Stage 7 — Observability**

## **Work**

Logger

Metrics

Latency measurement

/metrics

## **Deliverable**

You can inspect how the server behaves internally.

---

# **Stage 8 — Testing**

## **Work**

unit tests

integration tests

concurrency tests

sanitizers

## **Deliverable**

Evidence that the server is correct and safe.

---

# **Stage 9 — Benchmarking**

## **Work**

wrk

1 worker

2 workers

4 workers

8 workers

Measure:

throughput

p50

p95

p99  
errors

### **Deliverable**

A real benchmark table and analysis.

---

## **Stage 10 — Documentation + Resume**

### **Work**

Finalize:

README  
architecture diagram  
design decisions  
benchmark results  
screenshots/output  
resume bullets

### **Deliverable**

A polished GitHub project.

---

## **76. Git Strategy**

Use Git throughout development.

Do not write everything and make one giant commit.

Use meaningful commits such as:

initial CMake project setup  
implement TCP socket server  
add HTTP response serialization  
implement HTTP request parser  
add request validation  
implement router

- add GET and POST handlers
- implement thread pool
- add synchronized task queue
- add atomic server metrics
- implement graceful shutdown
- add structured logging
- add unit tests
- add integration tests
- add concurrency tests
- add benchmark scripts
- document architecture

This also makes the GitHub repository look like a real engineering project.

---

## 77. README Requirements

Your README should eventually contain:

### 1. Overview

What the server is.

### 2. Features

What it supports.

### 3. Architecture

Diagram and explanation.

### 4. Request Lifecycle

Explain the complete request path.

### 5. Thread Pool

Explain the worker/task model.

## **6. Synchronization**

Explain mutexes, condition variables, and atomics.

## **7. Error Handling**

Explain failure cases.

## **8. Testing**

Explain unit/integration/concurrency tests.

## **9. Benchmarking**

Show actual results.

## **10. Build**

Explain CMake commands.

## **11. Run**

Explain how to start the server.

## **12. Example Requests**

Show curl commands.

## **13. Design Decisions**

Explain important architectural choices.

## **14. Future Work**

Only after the core project is complete.

---



## 78. Final Resume Version

Once the project actually satisfies the requirements, the project could eventually be represented on your resume approximately like this:

### **Multithreaded HTTP Server** | C++, POSIX Sockets, CMake, Linux/macOS

- Built an HTTP/1.1 server from scratch in C++ using POSIX TCP sockets, implementing request parsing, routing, GET/POST handling, response serialization, and robust HTTP error handling.
- Designed a custom thread pool with a synchronized task queue using mutexes and condition variables to process concurrent client connections with bounded worker threads.
- Implemented graceful signal-based shutdown, thread-safe runtime metrics, structured request logging, and automated unit/integration tests for concurrent server behavior.
- Benchmarked single-threaded and multi-worker configurations under concurrent workloads, analyzing throughput and p95/p99 latency.

Again, these bullets should only be used once the implementation actually supports the claims.

---

## 79. What We Are Intentionally NOT Building Yet

For clarity, stop when the Core + Resume-Worthy system is complete.

Do **not** feel that the project is unfinished because it doesn't have:

- ✗ LRU cache
- ✗ Static file server
- ✗ Keep-alive
- ✗ HTTPS
- ✗ HTTP/2
- ✗ HTTP/3
- ✗ Authentication
- ✗ Database
- ✗ Reverse proxy
- ✗ Distributed deployment
- ✗ Advanced configuration

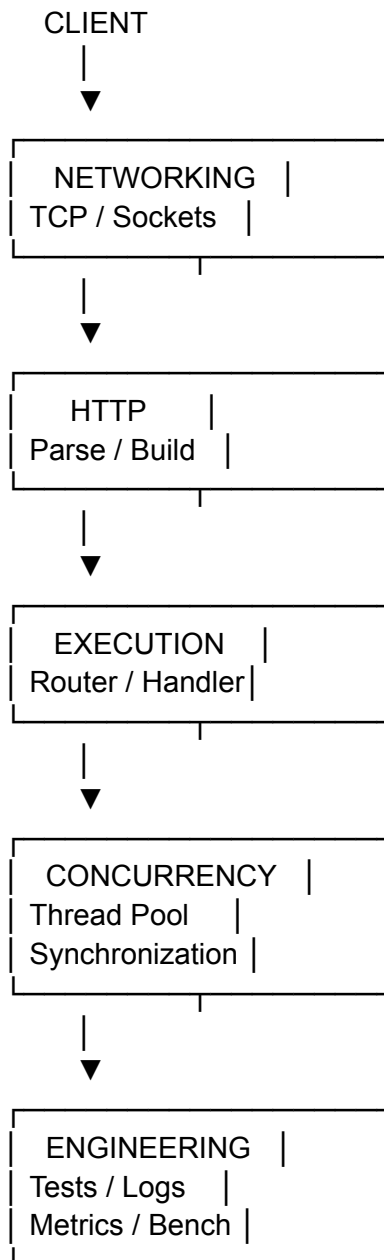
Those are **future extensions**, not missing requirements.

The project is complete when the defined scope is complete.

---

## 80. The Final Mental Model

The entire project can be remembered as five layers:



If you understand these five layers deeply, you understand the project.

---

# 81. The Actual Starting Point

Do **not** start by implementing the entire architecture.

Your first implementation target is only:

C++ program  
↓  
Create TCP socket  
↓  
Bind to port 8080  
↓  
Listen  
↓  
Accept connection  
↓  
Receive bytes  
↓  
Send bytes  
↓  
Close connection

Once that works, we move to HTTP.

Then routing.

Then concurrency.

Then thread pool.

Then robustness.

Then testing.

Then benchmarking.

That order is intentional.

The final project may look sophisticated, but you will have built it one understandable layer at a time.

---

# Final Target

At the end of the Core + Resume-Worthy implementation, you should be able to answer all of these without memorizing a script:

## Networking

What happens when a client connects to your server?

## TCP

What does `listen()` do? What does `accept()` return?

## HTTP

How do you determine where the HTTP headers end and the body begins?

## Parsing

What happens when the request is malformed?

## Concurrency

Why can't multiple workers simply modify shared state?

## Thread Pool

Why use a condition variable?

## Synchronization

When would you use an atomic instead of a mutex?

## Architecture

Why separate the parser, router, server, and thread pool?

## Shutdown

What happens to worker threads when Ctrl+C is pressed?

## Performance

Why doesn't doubling the number of workers necessarily double throughput?

## Testing

How did you verify that your server is actually thread-safe?

## Engineering

What bottleneck did your benchmark reveal?

If you can answer those questions using your **own implementation and measurements**, this project has achieved its purpose.

The goal is not simply to have a GitHub repository containing a C++ HTTP server.

The goal is to have a project where you can confidently say:

**“I built a concurrent network server from the TCP layer upward, and I understand the design decisions behind every major component.”**

That is the standard we should aim for.